

Track 16 — Was Stripe uns sagt, und was wir davon gelesen haben

Datum: 2026-08-28 **Ausgangsstand:** 9cded73 (Track 15), 1238 Tests **Endstand:** 1270 Tests (32 neu), Typecheck grün, Lint grün

Auftrag und Ergebnis vorweg

Geprüft wurde die vollständige Stripe-Strecke: Webhook-Handler, Session-Erstellung, Auszahlungs-Cron, Abo-Lebenszyklus, Connect-Onboarding, Refund-Pfade. Die klassischen Fragen eines Payment-Audits sind hier **ohne Befund** geblieben, und das ist die wichtigste Nachricht des Tracks:

Geprüft	Ergebnis
Webhook-Signatur	verifiziert. Kein Secret → 500, keine Signatur → 400, falsche Signatur → 400. <code>stripe.webhooks.constructEvent</code> mit Rohtext, <code>runtime = 'nodejs'</code>
Preis aus dem Request	nirgends. Termin: <code>bookings.price_cents</code> . Miete: serverseitig aus <code>rental_equipment</code> gerechnet. Shop: <code>order_items.unit_price_cents</code> (Track 14). Die Success-Seite schickt zwar ein <code>amount</code> mit — die Route liest es nicht
Fremde Buchung/Bestellung bezahlen	blockiert. Alle drei Zweige filtern auf <code>customer_id / renter_id</code> der Session (Track 13/15)
Idempotenz doppelter Events	vorhanden. Alle drei Geldstrecken: Statusprüfung, CAS-Claim (<code>.neq('payment_status', 'paid')</code>), Auto-Refund bei echter Doppelzahlung
Replay eines Events	Signatur trägt Zeitstempel (Stripe-Toleranz), und jede Wiederholung läuft in dieselben Zustandsriegel. Ein Guthaben-System, das sich hochzählen ließe, existiert nicht
Refund: wer darf	<code>admin/super_admin (/api/admin/refund)</code> , Mieter/Vermieter/Admin (Miet-Storno). Beide erstatten voll oder gar nicht , beide verweigern nach erfolgtem Provider-Transfer
Connect-Kapern (Salon A → Seller B)	nicht möglich. <code>provider_stripe_accounts</code> wird ausschließlich über die <code>user_id</code> der Session gelesen und geschrieben; <code>provider_user_id</code> der Auszahlung entsteht im Webhook aus dem DB-Join, nicht aus Request-Metadaten

Gefunden wurden fünf Befunde einer anderen Art: An **fünf Stellen** wurde ein Stripe-Objekt nur zur Hälfte gelesen — ein Feld, das mitgeliefert wurde und nie angesehen wurde, oder eines, das wir hätten mitgeben müssen und nie mitgegeben haben.

Befunde

1 — P1: Eine Teilerstattung galt als vollständige Erstattung

Stripe schickt `charge.refunded` bei **jeder** Erstattung — auch bei einer teilweisen. Ob es eine volle war, steht in der Charge selbst (`amount_refunded` gegen `amount`, dazu das Flag `refunded`). Der

Handler hat keines dieser drei Felder angesehen:

```
case 'charge.refunded': {
  const charge = event.data.object as Stripe.Charge
  const paymentIntent = charge.payment_intent as string
  if (paymentIntent) {
    // payments → 'refunded'
    // rental_bookings → status 'cancelled', payment_status 'refunded'
    // platform_transactions → 'refunded'
    // bookings → status 'cancelled', payment_status 'refunded'
    // orders → cancelled + releaseStockForOrder()
  }
}
```

Eine Kulanz-Rückzahlung von 5 € auf eine Miete von 500 € — der einzige Weg, so etwas zu tun, ist das Stripe-Dashboard — hatte damit diese Folgen:

- Die Mietbuchung wurde **storniert**, obwohl 495 € bezahlt bleiben.
- `platform_transactions` ging auf `refunded`, und `payment_status` auf `refunded` — genau die beiden Felder, die `cron/rental-payouts` als Ausschluss liest. Der Anbieter bekam für diese Miete **nie** eine Auszahlung.
- Beim Termin verlor die Kundin ihren bestätigten Termin.
- Bei einer Bestellung ging die **gesamte** Ware zurück ins Regal, obwohl der Kunde sie behält.

Jetzt: vollständig erstattet wird wie bisher nachgezogen. Eine Teilerstattung ändert **keinen** Zustand — sie landet als `charge_partially_refunded` im Audit-Log, mit Betrag und erstattetem Anteil. Die anteilige Rückabwicklung ist eine kaufmännische Entscheidung mit einer Zahl darin (welcher Teil trifft die Provision, welcher den Anbieteranteil?); die trifft kein Handler von selbst. Damit daraus keine Auszahlung des vollen Anteils auf einen teilweise zurückgezahlten Betrag wird, greift Befund 2.

2 — P1: Eine Rückbuchung wurde ausgezahlt

`charge.dispute.created` kam im Webhook **überhaupt nicht vor**. Nach einem Chargeback blieb die Miete `confirmed / paid` und die Plattform-Transaktion `succeeded` — und das sind die einzigen Bedingungen, unter denen der Payout-Cron am Mietbeginn `provider_share_cents` an den Connect-Account überweist. Die Plattform hätte das Geld zurückgegeben **und** ausgezahlt.

Der Cron hat für seine Entscheidung nie die Charge gefragt, nur die eigenen Spalten:

```
if (rental.status === 'cancelled' || rental.payment_status === 'refunded') { skipped++; continue; }
...
const pi = await stripe.paymentIntents.retrieve(tx.stripe_payment_intent_id)
const chargeId = typeof pi.latest_charge === 'string' ? pi.latest_charge : pi.latest_charge?.id
```

Der PaymentIntent wurde also geholt — nur um die Charge-ID daraus zu ziehen. `amount_refunded` und `disputed` standen zwei Zeilen weiter zur Verfügung.

Jetzt: `expand: ['latest_charge']`, und die Auszahlung wird ausgesetzt, wenn die Charge (teil-)erstattet oder angefochten ist. Beides landet sichtbar in `errors[]` der Cron-Antwort, statt still übersprungen zu werden. `charge.dispute.created / .closed` schreiben zusätzlich einen Audit-

Eintrag — **ohne** den Zustand umzuschreiben: `payments.status` und `platform_transactions.status` kennen live kein Wort für „angefochten“ (CHECK: `pending|succeeded|failed|refunded`), und eines zu erfinden hieße, eine Rückbuchung als Erstattung auszugeben.

3 — P1: Termin- und Shop-Zahlungen hatten keinen Zahler

`createRentalCheckout` schreibt `user_id` in die Session-Metadaten. `createBookingCheckout` und `createProductOrderCheckout` **nicht**:

```
metadata: { booking_id: params.bookingId, type: 'booking_payment' },
metadata: { order_id: params.orderId, order_number: params.orderNumber, type: 'product_order' }
```

Der Webhook liest `meta.user_id` an zwölf Stellen. Für Termin und Bestellung war der Wert in **jedem** Request `undefined`:

- `payments.user_id` blieb bei jeder Termin- und jeder Shop-Zahlung leer — eine Zahlung ohne Zahler.
- Die Audit-Einträge `payment_completed` und `product_order_paid` waren kontenlos.
- `if (meta.user_id) { await createNotification(...) }` war immer falsch: die Nachricht „**Zahlung bestätigt**“ ist für Termine nie verschickt worden. Vollständig verdrahtet, nie ausgelöst. Bei der Bestellung hat es nur überlebt, weil dort zusätzlich auf `order.customer_id` zurückgefallen wird.

Jetzt: beide Sessions tragen `user_id`, und alle drei Zweige lösen den Zahler zusätzlich über die DB-Zeile auf (`booking.customer_id`, `order.customer_id`, `rental.renter_id`) — damit auch die Sessions funktionieren, die beim Deploy dieser Änderung schon offen waren.

4 — P1: Zwei Klicks, zwei Connect-Konten, keine Auszahlung mehr

```
const { data: existing } = await supabase
  .from('provider_stripe_accounts')
  .select('...').eq('user_id', userId).maybeSingle()
let accountId = existing?.stripe_account_id
if (!accountId) { /* neuen Stripe-Express-Account anlegen */ }
```

Der Fehler neben `data` wurde nicht angesehen. Gibt es zu einem Anbieter **mehrere** Zeilen, antwortet PostgreSQL mit PGRST116 und `data: null` — für den Code sah das aus wie „noch kein Konto“, und jeder weitere Aufruf legte bei Stripe ein **weiteres** Express-Konto an. Zwei Zeilen entstehen ohne Zutun: zwei parallele Klicks auf „Stripe verbinden“.

Schlimmer ist die andere Seite: `cron/rental-payouts` liest dieselbe Tabelle mit demselben `.maybeSingle()`, bekam denselben Fehler, las ihn als „kein Connect-Account“ — und dieser Anbieter wurde ab da **nie wieder ausgezahlt**, ohne dass irgendwo etwas stand.

Der UNIQUE-Index dagegen (`uq_provider_stripe_user`) steht in Migration

`20260705_rental_booking_constraints.sql`. Ob sie live angewendet ist, lässt sich von hier nicht prüfen — es gibt keinen DB-Zugang für Agents. Der Riegel fällt deshalb im Code: Lesefehler und Mehrdeutigkeit führen zu **keinem** neuen Stripe-Konto (409/503) und zu **keiner** Auszahlung; beides wird sichtbar gemacht statt stillschweigend übersprungen. Auch `GET` meldet einen Lesefehler jetzt

als 503, statt „nicht verbunden" zu behaupten — sonst bietet die Oberfläche ein Onboarding an, das ein zweites Konto anlegen würde.

5 — P2: Ein zweites Abo auf dasselbe Konto war nichts, was etwas verhindert hätte

Die Stufe steht in `salons.subscription_tier` — **ein** Wert, egal wie viele Abos dahinter laufen. Die Tabelle `provider_subscriptions` existiert, wird vom Produktivcode aber nirgends beschrieben; es gab also gar keinen Ort, an dem „hier läuft schon ein Abo" hätte stehen können. Zwei Tabs, zweimal geklickt, und der Anbieter zahlt ab sofort zweimal im Monat. Kündigt er eines davon, meldet Stripe `customer.subscription.deleted` und `handleSubscriptionChange` stuft ihn auf die kostenlose Stufe zurück — während das zweite Abo weiterläuft und weiter abgebucht wird.

Dazu kam: `createSubscriptionCheckout` übergab immer `customer_email`, nie `customer`. Stripe legt dann bei **jedem** Checkout einen neuen Kunden an, und der Webhook überschreibt `profiles.stripe_customer_id` mit dem zuletzt entstandenen — das ältere Abo ist über den Rückfallweg nicht mehr auffindbar.

Jetzt: der Checkout fragt Stripe nach den Abos des bekannten Kunden und lehnt mit 409 ab, solange dort eines läuft (`entitlementForStatus ≠ revoked`, also auch während der Mahnkette — `past_due` wird weiter abgerechnet). Eine vorhandene Kundennummer wird als `customer` durchgereicht. Ohne Kundennummer wird Stripe gar nicht erst gefragt; ein Lesefehler am Profil führt zu 503 statt zu einem ungeprüften Kauf.

6 — P2: Nachzahlung an einen gesperrten Salon

Track 15 hat den Salon-Riegel auf die Strecken gelegt, auf denen eine Verpflichtung **entsteht** (`createBooking`, `POST /api/rental-bookings`). Die **Nachzahlung** einer bereits bestehenden Buchung lief daran vorbei: ein Termin oder eine Miete, angelegt vor der Sperre, ließ sich über `POST /api/stripe/checkout` unverändert bezahlen. Beim Termin bleibt das Geld auf dem Plattformkonto — bei der Miete nicht: der Payout-Cron überweist den Anbieteranteil am Mietbeginn an genau den Anbieter, den die Plattform angehalten hat.

Jetzt prüfen beide Zweige `salonAcceptsBusiness`. Beim Miet-Zweig steht die Prüfung bewusst **vor** dem Verfallenlassen der alten Checkout-Session — sonst hätte eine abgelehnte Anfrage dem Mieter die einzige Session genommen, die er noch hatte.

7 — P2: Ein Betrag, den Stripe nicht einziehen kann, wurde 500

`calculatePrice` deckelt einen Rabatt ausdrücklich bei 0 — ein Promo-Code mit 100 % ergibt also einen Termin über 0 €. `createBookingCheckout` reicht das als `unit_amount: 0` an Stripe weiter, die Session-Erstellung wirft, und der `catch` am Ende der Route macht daraus „Interner Fehler" (500). Die Kundin las einen Serverfehler, wo eine Erklärung hingehört.

Jetzt antworten alle drei Zweige mit 409 und einem Satz, wenn der Betrag nicht positiv ist. Ein Mindestbetrag wird **nicht** erfunden — geprüft wird nur, was zweifelsfrei nicht zahlbar ist.

Ohne Befund geprüft

- **Webhook-Signatur** — siehe Tabelle oben; drei bestehende Tests decken es ab.

- **Preisquelle aller drei Strecken** — kein Betrag stammt aus dem Request.
 - **Cross-Tenant-Checkout** — Termin, Miete und Bestellung filtern jeweils auf das Konto der Session.
 - **getOrCreateSalonSeller** — `sellers` hat `UNIQUE(user_id, seller_type)` ; ein Konto kann nie zwei Salon-Verkäufer haben. Gelesen und geschrieben wird ausschließlich mit der `userId` der Session, ein fremder Verkäufer ist darüber nicht erreichbar. Angepasst wurde nur die Fehlerbehandlung: `.single()` mit ignoriertem Fehler ließ einen DB-Aussetzer wie „gibt es noch nicht“ aussehen, der folgende Insert lief in die Unique-Verletzung, und der Anbieter las „Produkt konnte nicht angelegt werden“ statt „bitte gleich noch einmal“.
 - **Cron-Autorisierung** — `isAuthorizedCron` mit `timingSafeEqual` , fehlendes `CRON_SECRET` sperrt (Track 12). Jetzt zusätzlich als Test festgehalten.
 - **Auszahlungs-Dedupe** — Idempotency-Key am Transfer, Backstop-Abfrage gegen bereits transferierte Transaktionen derselben Miete, UNIQUE-Index `uq_pltx_rental_succeeded` .
 - **Stripe-Rückkehr-URLs** — seit Track 12 aus `src/lib/app-origin.ts` , nicht aus dem Origin-Header.
-

Was NICHT gemacht wurde

- **Keine Migration angewendet.** `uq_provider_stripe_user` bleibt offen; der Schutz aus Befund 4 wirkt im Code, unabhängig davon.
 - **Keine Beträge erfunden.** Weder ein Mindestbetrag, noch eine Aufteilung einer Teilerstattung, noch ein Status für „angefochten“.
 - **Keine Auszahlung nachgeholt.** Anbieter, die durch Befund 1 oder 4 eine Auszahlung verloren haben, bekommen sie nicht rückwirkend: welche das sind, ist von hier aus nicht lesbar (kein DB-Zugang), und ein Transfer ist kein Nebeneffekt eines Härte-Tracks.
-

Operative Folge für yusuf

1. **Teilerstattungen im Stripe-Dashboard sind ab jetzt folgenlos für die Buchung** — sie stornieren nichts mehr. Wenn eine Teilerstattung *auch* stornieren soll, muss der Storno zusätzlich in ChairMatch ausgelöst werden. 2. **Der Payout-Cron zahlt eine Miete nicht mehr aus, wenn an ihrer Zahlung irgendetwas erstattet oder angefochten wurde.** Solche Fälle stehen in der Antwort des Crons unter `errors[]` und brauchen eine Entscheidung von Hand. 3. **Anbieter mit zwei Connect-Konten werden nicht mehr ausgezahlt**, sondern in `errors[]` benannt. Falls das auftritt: im Stripe-Dashboard klären, welches Konto gilt, und die überzählige Zeile in `provider_stripe_accounts` entfernen.

Zahlen

	vorher	nachher
Tests	1238	1270 (+32)
Testdateien	68	69
Typecheck	grün	grün

Neue Testdatei: `src/app/api/cron/__tests__/rental-payouts.e2e.test.ts` (8) — der Auszahlungs-Cron hatte bis hierher **keine** Testabdeckung, obwohl er die einzige Stelle ist, an der ChairMatch Geld aus der Hand gibt.